

Async/Await Pattern

Problemstellung



- Code ist unglaublich langsam
- Das gilt insbesondere für folgende Zugriffe



Datebank



Netzwerk



Lösung

async / await Keywords

- Ermöglicht Parallelisierung der Codeausführung
- Kann jedem Codeteil hinzugefügt werden
- **async** Methoden müssen einen **Task**, **Task<T>** oder **ValueTask<T>** zurück geben
- **await** erzeugt eine sog. "Coroutine" (nicht zwangsläufig einen neuen Thread)
- Drawback: schwieriger zu debuggen



3

- das Keyword `async` hat erstmal keine Auswirkung und ist nur eine Info für den Entwickler
- `Task` ist erstmal nur ein "fancy Wrapper" für den eigentlichen Return Type
- Methodenaufrufe mit `await` machen das Stück Code `async`. Es weist das Programm an eine Coroutine zu erzeugen. Alle anderen Lines of Code ohne `await` werden synchron abgearbeitet

Warum `ValueTask<T>` statt `Task<T>`?

<https://stackoverflow.com/questions/43000520/why-would-one-use-taskt-over-valuetaskt-in-c>

The default choice for any asynchronous method should be to return a `Task` or `Task<TResult>`.

Only if performance analysis proves it worthwhile should a `ValueTask<TResult>` be used instead of `Task<TResult>`.

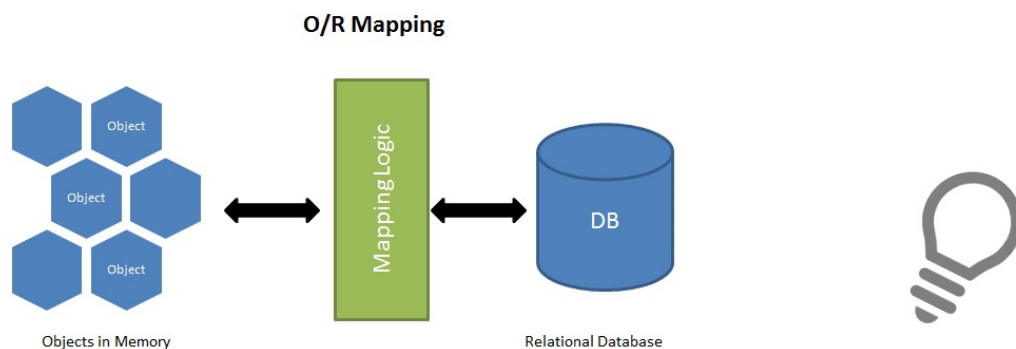


Entity Framework

SQL Server, Connection String, EF, Scaffolding, Lazy Loading (Include), LINQ, Paging, HttpClient Generische Listen, Annotations

Object-Relational Mapping

- *Object-Relational Mapping (ORM)* ist eine Technik, die das Abfragen und Ändern von Daten einer Datenbank mittels Objekt-Orientierten Paradigmen ermöglicht.





Warum Objektrelationales Mapping?

- **Problem:**

- Relationale Modelle und Objekt Modelle arbeiten schwierig vereinbar
- Tabellarisches Datenbankformat vs. Objektdarstellung

- **Lösung:**

- ORM: Bibliothek, die ein Objekt-Relationales Mapping implementiert
- Kümmt sich um das Mapping zwischen tabellarischem Format und Objektdarstellung
- ORM spart Entwicklungszeit!



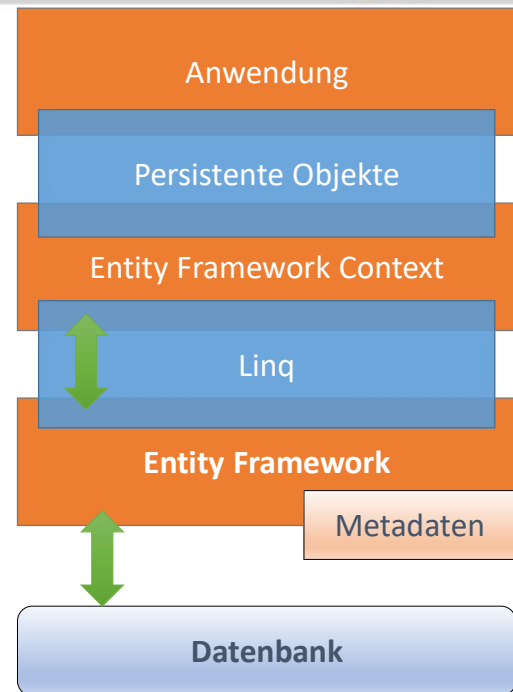
Gängige ORM Mapper für ASP.NET Core

- **Entity Framework Core**
 - <https://github.com/aspnet/EntityFrameworkCore>
- **Dapper Micro ORM**
 - <https://github.com/StackExchange/Dapper>



Entity Framework Core

- O/R Mapping Framework
- Leichtgewichtig, Erweiterbar, Cross-Plattform Version von EF6
- Übersetzt LINQ in SQL
- Ermöglicht es, Daten nur bei Bedarf zu laden (lazy loading)
- Unterstützt mehrere relationale Datenbanken (NoSQL noch nicht)
- Code First & Database First Ansatz



<http://www.entityframeworktutorial.net/efcore/>

<https://www.learnentityframeworkcore.com/>



Entity Framework Core Schlüsselbegriffe

- **Data Annotations** = definieren Primary und Foreign Keys, Required Fields, ... in Entity Klassen
- **DbContext** = eine Session mit einer Datenbank und ermöglicht das Abfragen und Speichern von Instanzen von Entitäten
- **Seeding** = Demodaten bereitstellen
- Environment Variablen für geheime Daten verwenden
 - ConnectionStrings, Username + Passwort, ...





Entity Framework Core Schlüsselbegriffe

- **Code First Ansatz:**

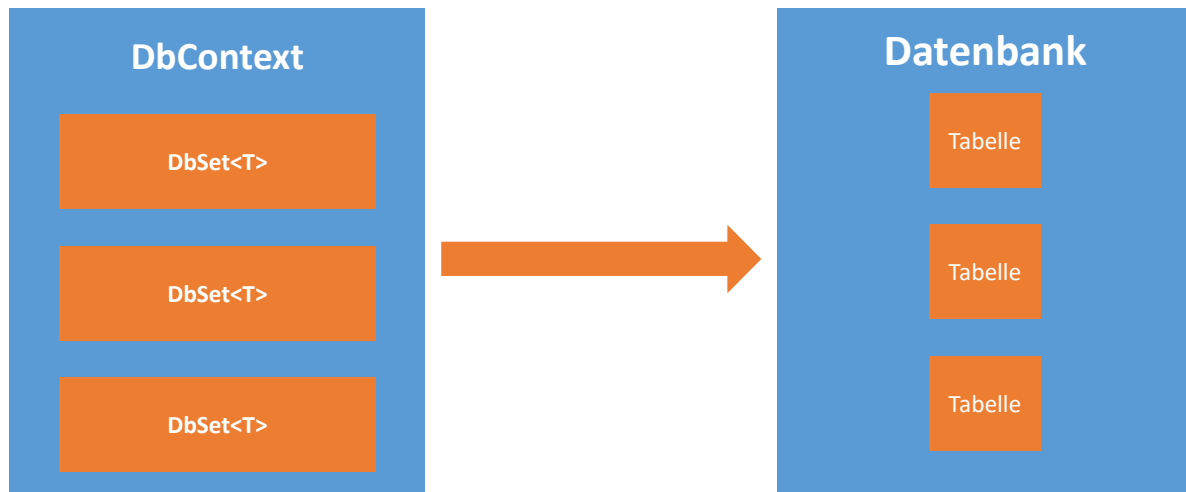
- Code => DB
- Geeignet für neue Entwicklungen
- Keine Datenbank Programmierung in SQL notwendig

- **DB First Ansatz:**

- DB => Code
- Erstellung der Model Klassen aus der Datenbank
- Geeignet für bereits bestehende Datenbanken



Object-Relational Mapping in EF Core



Zuständig für: DB Connection, Data Operations, Change Tracking, Model Building, Data Mapping, Object Caching, Transaction Management



Database First

- Installieren der NuGet Packages:
 - Microsoft.EntityFrameworkCore.SqlServer
 - Microsoft.EntityFrameworkCore.Tools
 - Microsoft.VisualStudio.Web.CodeGeneration.Design

- DbContext generieren mittels PM Console:

```
Scaffold-DbContext "Server= ServerName;Database= DbName;  
Trusted_Connection=True;" Microsoft.EntityFrameworkCore.SqlServer  
-OutputDir Models -t dbo.Car
```

<https://docs.microsoft.com/en-us/ef/core/get-started/aspnetcore/existing-db>



DbContext Konfiguration Startup.cs

- ConfigureServices Methode
- DI Model per AddDbContext
- DB Typ per DbContextOptions
 - DB Provider (UseSqlServer, UseSqlite, UseMySQL, UseInMemory ...)
 - Connection String (appsettings.json)
 - Optionale Behavior und Selector Einstellungen

```
services.AddDbContext<ApplicationDbContext>(options =>  
    options.UseSqlite(  
        Configuration.GetConnectionString("DefaultConnection")));
```



DbContext Konfig

- Per Tabelle/Objekt ein DbSet<TEntity>
- Konstruktor zwei Optionen
 - mit DI Options Injection
 - Leer->OnConfiguring (Fluent Api)

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    { }

    public DbSet<Auto> Autos { get; set; }
}
```



DbSet Konfiguration

- Data Annotation Attributes in Model Klassen
- Fluent API => Konfiguration mittels modelBuilder Methoden
 - `Microsoft.EntityFrameworkCore.ModelBuilder`
 - Model Konfiguration, Property Konfiguration, ...

```
public class ApplicationDbContext : DbContext
{
    // DbSets

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Konfigurationen
    }
}
```



Connection String für DB Provider

- Dev vs production
- Connection Strings in appsettings.json:

```
"ConnectionStrings": {  
  "MSSQL": "Server=Name;Database=Name;Integrated Security=True;Trusted_Connection=True;",  
  "MySQL": "Server=localhost; Database=Name; Uid=user; Pwd=password",  
  "SQLite": "Data Source=Name.db",  
  ...  
}
```




Model First & Migrations

- Datenbank anhand Model Klassen

Package Manager Anweisung	Verwendung
add-migration <i>MigrationName</i>	Erstellen einer Migration
update-database	Update von Datenbank Schema auf Basis der Migration
remove-migration	Entfernt letzte Migration
script-migration	Erstellung eines SQL Scripts auf Basis der letzten migration
update-database -migration Name	Rücksprung zu spezifischer Migration



Konventionen

- ID, TabelleID
- Foreign Key FremdTabelleID
- Virtuelle Property
 - List<ClientTabelle>
- Attribute
 - Key
 - DatabaseGenerated(DatabaseGeneratedOption.Identity)



Erste Schritte

- Lesen
 - Generische Liste
- Schreiben
 - Entität
 - SaveChanges
- Plain SQL
 - Entity FromSQL
 - Database.ExecuteSqlCommandAsync



LINQ Basics

- Typisierte Daten Aggregationen
 - Mehrzeilig
 - Einzeilig -> auch mehrere Zeilen
- Where
 - Find Key
- Group by
- Sort
- Select
 - New oder Anonym
- Join

```
var queryLondonCustomers = from cust in customers
where cust.City == "London"
select cust;
```

```
var studentNames = studentList.Where(s => s.Age > 18)
    .Select(s => s)
    .Where(st => st.StandardID > 0)
    .Select(s => s.StudentName);
```



Lazy Loading

- Verknüpfungen werden nicht geladen
 - Count Children
- Include
 - String
 - Besser Lambda Ausdruck



Globale AbfrageFilter

- OnModelCreating
 - z.B. Userrechte
 - HasQueryFilter
- IgnoreQueryFilters

<https://docs.microsoft.com/de-de/ef/core/querying/filters>



Tracking generiertes SQL

- SQL Profiler
 - AbfrageTags
 - TagWith("This is my spatial query!")
- Logging

<https://docs.microsoft.com/en-us/ef/core/miscellaneous/logging>